

ELI — Commands and Installers

ELI v2.1.51

August 2026

Contents

ELI v2.0 — Commands & Installers Reference	1
1. The one-click installer (recommended for everyone)	1
2. Full install from source (developers)	2
3. Launching ELI	3
4. Models	3
5. Alternative & advanced install paths	4
6. Building release packages (maintainer)	4
7. Health checks & maintenance	5
8. Uninstall	5

ELI v2.0 — Commands & Installers Reference

Updated for v2.1.51 (August 2026). The primary way to install ELI is now the **prebuilt installers on GitHub Releases** — ELI-Setup-<v>.exe (Windows), the .dmg (macOS, Apple Silicon), the .AppImage (Linux). The Linux AppImage and Windows installer are built and launch-tested in CI; the macOS .dmg is built on a Mac and provided best-effort (not verified in CI). First launch offers GPU acceleration (NVIDIA CUDA / AMD Vulkan; Apple Metal is built in) and a starter model sized to your hardware. Data lives in a per-user ELI_v2 folder and survives upgrades; --fresh-start resets it. Everything below remains valid for **source installs** (clone + install.sh) and the classic portable tarball.

Every install path and command in one place. Copy-paste ready. Run everything from inside the ELI folder unless noted.

Canonical version: **2.1.23**. Best-tested platform: **Linux x86_64 + NVIDIA**. Windows, macOS, and AMD are coded for and ship installers, but expect rough edges.

1. The one-click installer (recommended for everyone)

This is the gentlest path — it does the whole job and opens ELI at the end. Safe to run more than once.

From a downloaded release (no git, no build)

Get ELI_v2-2.1.51-linux-portable.tar.gz from [GitHub Releases](#), then:

```
tar -xzf ELI_v2-2.1.51-linux-portable.tar.gz
cd ELI_v2-2.1.51-linux-portable
chmod +x ELI_Setup.sh
./ELI_Setup.sh
```

ELI_Setup.sh runs the same 8-step one-click setup as scripts/eli_setup.sh:

1. Welcome
2. Python check (needs Python 3.10+)
3. Python environment + dependencies (install.sh --yes --auto-model)
4. Starter model pack (GitHub asset restore, or a hardware-sized auto-download)
5. Local database
6. Memory embedder + voice models
7. App-menu icons (ELI v2.0, ELI Server, ELI Setup)
8. Opens the graphical setup wizard and launches ELI

From a git clone

```
git clone https://github.com/ShadowESC95/ELI_v2.0.git
cd ELI_v2.0
./scripts/eli_setup.sh
```

The absolute-easiest Linux path: the AppImage

Get ELI_v2-2.1.51-x86_64.AppImage from Releases:

```
chmod +x ELI_v2-2.1.51-x86_64.AppImage
./ELI_v2-2.1.51-x86_64.AppImage
```

First double-click installs ELI to ~/.local/share/ELI_v2 and runs setup once; every launch after that opens ELI directly.

2. Full install from source (developers)

Linux / macOS

```
git clone https://github.com/ShadowESC95/ELI_v2.0.git
cd ELI_v2.0
bash install.sh          # interactive: system report → plan → install → pick model
./scripts/eli_launch.sh  # launch the desktop app
```

install.sh flags (combine as needed):

Flag	Effect
--yes / -y	No prompts — use detected defaults (CI / piped installs)
--cpu-only	Force the CPU build (ignore GPU)
--gpu	Force the GPU build even if none is auto-detected
--install-cuda / --cuda	Best-effort install the CUDA toolkit (nvcc), then rebuild llama-cpp with CUDA
--auto-model	Download one model sized to your VRAM after install
--model=KEY	Download a specific model (e.g. --model=qwen2.5-7b)
--no-model	Never download a model (also skips embedder/voice) — fully offline install
--latest	Use version ranges instead of the frozen reproducible lock
--skip-torch	Skip PyTorch (smaller install; disables self-training)

Examples:

```
bash install.sh --yes --auto-model           # hands-off, with a model
bash install.sh --cpu-only --no-model        # minimal, no GPU, add a model later
bash install.sh --install-cuda --model=phi-4 # CUDA toolkit + a specific model
```

Windows

```
install.bat           :: normal – CUDA install from the frozen lock + GPU verify
install.bat /cpu      :: CPU-only
install.bat /cuda      :: also auto-install the CUDA toolkit (winget) if missing
install.bat /latest    :: version ranges instead of the frozen lock
```

Or call the PowerShell installer directly:

```
powershell -ExecutionPolicy Bypass -File install.ps1 [-CpuOnly] [-Gpu] `
  [-InstallCuda] [-Yes] [-AutoModel] [-NoModel] [-Model qwen2.5-7b] [-Latest]
```

Launch with eli.bat.

Developer editable install (pip)

```
python3 -m venv .venv && . .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[full]"
python -m eli                # GUI
python -m eli --headless      # terminal REPL
```

Android / Termux (headless — server + CLI, no GUI)

```
bash scripts/install_android.sh
```

3. Launching ELI

```
./scripts/eli_launch.sh           # desktop app (GUI) – default
./scripts/eli_launch.sh gui       # same, explicit
./scripts/eli_launch.sh serve     # API + web-app server (this machine only)
./scripts/eli_launch.sh serve --lan # server exposed to your home network
./scripts/eli_launch.sh both --lan # server in background + desktop app
./eli.sh                          # shortcut for the desktop app
.venv/bin/python -m eli           # desktop app via module
.venv/bin/python -m eli --headless # text-only terminal REPL
```

Headless slash commands: /status · /mode · /reset · /help · /quit

The web / phone server directly:

```
./scripts/eli_serve.sh           # 127.0.0.1 only → http://127.0.0.1:8081/
./scripts/eli_serve.sh --lan      # home network → prints a token-protected URL + QR
./scripts/eli_serve.sh --port 9000 # custom port
./scripts/eli_serve.sh --lan --token MYSECRET # use a specific access token
```

4. Models

```
.venv/bin/python -m eli.core.model_download --list # show the catalog
.venv/bin/python -m eli.core.model_download --auto # one best-fit for your VRAM
```

```
.venv/bin/python -m eli.core.model_download --choose # multi-select menu (pick any number)
.venv/bin/python -m eli.core.model_download --aux # just the memory embedder (~85 MB)
```

Bring your own: drop any chat/instruct .gguf into models/ — ELI finds it and fits it to your hardware. Vision models also need their matching mmpoj GGUF alongside.

Voice weights (if not fetched during install):

```
.venv/bin/python -m eli.runtime.voice_assets
```

Restore the model/voice pack from GitHub (tag local-assets-v2.1, needs gh):

```
gh auth login
./RUN_ELI.sh --with-github-assets
# or, without gh, after downloading assets manually:
python3 scripts/restore_github_asset_files.py --from-dir /path/to/downloaded/assets
```

5. Alternative & advanced install paths

Path	Command	When to use
One-click run (portable)	./RUN_ELI.sh	Daily launch of a portable install
Classic portable install	./INSTALL_ELI.sh then ./RUN_ELI.sh	Portable package, manual two-step
Safe Linux install	bash scripts/safe_install_linux.sh	Conservative install with extra checks
Add the eli terminal command	bash scripts/install_eli_command.sh	Get eli on your \$PATH (~/local/bin/eli)
Install app-menu icons only	bash scripts/install_desktop_apps.sh	Re-add ELI / ELI Server / ELI Setup launchers
Repo venv runner	bash scripts/run_eli_repo_venv.sh	Run against the repo's own .venv
Purge a legacy install	bash scripts/purge_legacy_eli.sh	Clean up an older ELI before reinstalling

If your shell cached an old eli command after reinstalling: hash -r.

6. Building release packages (maintainer)

```
bash scripts/build_v2_release.sh # portable Linux tar.gz
bash scripts/build_v2_release.sh --with-assets # + bundled models (very large)
bash scripts/build_grandma_release.sh # portable tar.gz + AppImage + checksums
bash packaging/linux/build-appimage.sh # AppImage only (from the portable build)
bash scripts/package_eli_release.sh # wheel + sdist
bash build_packages.sh wheel appimage windows-lean # pick targets
```

Windows Setup.exe (run on a Windows PC with Inno Setup 6):

```
bash build_packages.sh windows-lean
powershell -ExecutionPolicy Bypass -File packaging/windows/build-windows.ps1 -Version 2.1.23
```

A signed/notarized macOS .dmg must be built on a Mac. Large model/voice binaries ship separately as GitHub Release assets (over Git's 100 MB blob limit). Full publish steps: [RELEASE.md](#).

7. Health checks & maintenance

```
bash run_tests.sh           # run the test suite
bash eli_diagnose.sh        # system diagnosis report
.venv/bin/python -m eli.tools.mic_diag # microphone diagnostics (voice input)
.venv/bin/python -m eli.core.init_data  # (re)build the local databases
```

Reading the licence

ELI is **source-available**, not open-source: PolyForm Internal Use License 1.0.0. Run and modify it for your own personal or internal use; you may not redistribute, host, sublicense or sell it. The terms are reachable identically from every download — installed app, portable folder, AppImage or .app:

```
eli --license           # source install / portable tarball
./ELI_v2-*.AppImage --license
ELI.exe --license       # Windows (the Setup.exe also shows it during install)
```

The files ship alongside the app too: LICENSE, NOTICE, THIRD_PARTY_NOTICES.md (dependency licences, incl. the PySide6 LGPL note) and models/MODEL_LICENSES.md (model and voice terms).

8. Uninstall

ELI lives entirely inside its own folder — nothing spreads across your system:

```
rm -rf /path/to/ELI_v2.0
```

For an AppImage install, also remove ~/.local/share/ELI_v2. If you added app-menu icons, delete the .desktop files from ~/.local/share/applications/.

ELI v2.0 — © 2026 Jason Fitzgibbon Bridgeman. Source-available under the PolyForm Internal Use License 1.0.0. Questions: jaybridgeman0095@gmail.com